前置知識與本章目標

- 前置知識為 Array ADT、索引區間與迴圈不變量。
- 能以操作契約區分 LIFO 堆疊和 FIFO 佇列，並處理空容器、滿容器與環狀回繞。
- 能以狀態追蹤解釋迷宮搜尋與運算式求值，不把資料結構選擇誤認為唯一演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stack ADT 的 LIFO 契約

- 抽象狀態為序列 $[a_0,\ldots,a_{n-1}]$，頂端在末端。`push(x)` 把 `x` 接在末端。
- `top()` 回傳最後一個元素但不改狀態。`pop()` 移除末端元素，前提是非空。
- 在空堆疊呼叫 `pop` 的行為由介面定義。可回傳 optional、例外或錯誤碼，不能默默讀取無效記憶體。
- 表示為動態陣列時，`top`、`pop` 為常數時間，`push` 為攤銷常數時間。固定容量版本另有滿溢結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

以 n 表示堆疊頂端位置

- 不變量為 `0≤n≤capacity`，有效元素位於 `[0,n)`。頂端是 `A[n-1]`，只有 `n>0` 才能存取。
- `push(x)` 先確保有容量，再寫 `A[n]`，最後增加 `n`。若配置失敗，不應把 `n` 提前增加。
- `pop` 先檢查 `n>0`，再移除最後一個有效物件。泛型 C++ 實作還要處理析構，不只是改索引。
- 以 `n=0` 表示空狀態可避免以無號索引表示 `top=-1` 的問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

陣列堆疊：完整操作與物件生命週期

設 `storage` 是容量 C 的未初始化儲存，n 是活物件數。抽象序列等於 `storage[0..n)`；不能把「配置了 C 格」誤當成「已建構 C 個 T」。

```
top(): require n>0; return const T& to storage[n-1]
push(T value):           // snapshot constructed before entering the body
    if n==C: grow_to(max(1,2*C))
    construct_at(storage+n, value)   // 成功後才提交 n
    n=n+1
pop():
    require n>0
    destroy_at(storage+n-1)
    n=n-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

以 `C=2,n=0` 依序 push(8)、push(3)、push(2)、pop()，狀態為 `(2,1,[8])`、`(2,2,[8,3])`、`(4,3,[8,3,2])`、`(4,2,[8,3])`。第三次 push 先把兩個既有值搬到新區，故單次可能 Θ(n)，不是最壞常數。從空開始倍增的整段 append 總成本 Θ(n)。

陣列堆疊：完整操作與物件生命週期 (續)

若使用泛型 T，搬移／複製與析構的成本需另計。要承諾擴容失敗時原狀不變，需可用的複製或 noexcept move 策略；任意會拋例外的 move 不能無條件給強例外保證。

本介面以push(T value)按值傳遞，限定可複製建構的T；value在進入本體前已是獨立物件。top()雖回const T&，push(top())會先複製目前值再grow。容量1、size1且內容[7]時，順序是建構value=7→搬原7到新區→釋放舊區→從value建構第2筆→size=2，得到[7,7]。若push收const T&，舊top的參數別名在釋放舊區後懸空；即使搬移是noexcept，也不會讓舊參照重新有效。

快照建構成本需額外計入，只有固定成本T才沿用常數攤銷結論；不可複製的T不支援這個push(top())複製用法。參數複製若失敗，本體尚未開始，size、capacity及既有物件不變。成功grow後若新元素建構失敗，本偽碼只保證size不增加、原值仍有效；capacity及舊參照可能已改變，不宣稱整個push皆有強例外保證。

Queue ADT 的 FIFO 契約

- 抽象序列同樣為 $[a_0,\ldots,a_{n-1}]$。`enqueue(x)` 加在末端，`dequeue()` 移除開頭。
- 若每次移除開頭都把陣列左移，單次 `dequeue` 為 $\Theta(n)$。
- 改以環狀陣列儲存，可避免為保留邏輯順序而搬動所有剩餘元素。
- FIFO 描述的是回傳次序，不保證多執行緒安全，也不等於每個工作具有相同執行時間。

環狀佇列的表示不變量

- 令容量 `C>0`，儲存 `head` 和元素數 `size`。第 `k` 個邏輯元素在 `(head+k) mod C`。
- 空條件 `size=0`，滿條件 `size=C`。不需浪費一格區分 `head==tail` 的歧義。
- enqueue 寫入 `(head+size) mod C` 後增加 `size`。dequeue 讀 `head`，使 `head=(head+1) mod C`，再減少 `size`。
- 每次操作最壞 `O(1)`，前提是固定容量及常數時間元素移動。擴容時需依邏輯順序複製，單次為 `O(n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀追蹤

容量為 4，依序加入 `A,B,C`，移除兩次，再加入 `D,E,F`。

狀態	head	size	邏輯序列
加入 ABC	0	3	A,B,C
移除兩次	2	1	C
加入 D,E	2	3	C,D,E
加入 F	2	4	C,D,E,F

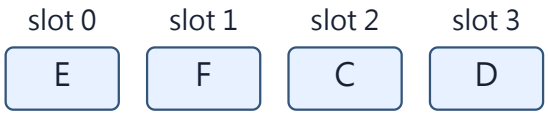
最後實體位置 0、1、2、3 依序存 `E,F,C,D`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

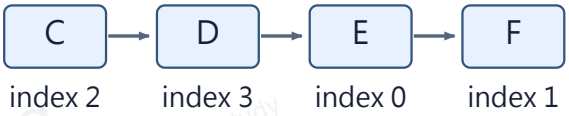
環狀追蹤 （續）

實體 [E,F,C,D] 與 FIFO [C,D,E,F]

實體槽位



邏輯順序 / 實體索引



head=2; size=4; C=4

indices: 2 → 3 → 0 → 1

slot(k)=(head+k) mod C

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀佇列完整操作與溢位邊界

以 head、size 區分空滿，容量 C>0。以下是固定容量版本；`try_enqueue` 遇滿回報失敗，不覆寫最舊值。回傳值的移動成本與例外政策由元素型別契約決定。

```
slot(k):
    require 0<=k<C
    return head+k if k<C-head else k-(C-head)
try_enqueue(x):
    if size==C: return full
    construct_at(A+slot(size),x)
    size=size+1; return success
front(): require size>0; return reference(A[head])
dequeue():
    require size>0
    value=read_or_move(A[head])
    destroy_at(A+head)
    head=0 if head==C-1 else head+1
    size=size-1
    return value
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀佇列完整操作與溢位邊界 (續)

分支式 slot 避免先算 head+k 溢位。呼叫時先確認未滿，故 k=size<C；出列最後一項後 head 可停在任一合法槽，不需強制歸零。正確性由 `(head+k) mod C` 的抽象映射驗證：入列只新增第 size 個邏輯值，出列後原第 k+1 個值成為新第 k 個值。兩者固定容量最壞 Θ(1)，並非只靠平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀擴容不可直接保留舊槽號

從 physical=`[E,F,C,D]`、head=2、size=4、C=4 擴成 C′=8。若只把四格複製至新陣列前四格卻沿用 head=2，新取模會把原應回繞的 E 指向尚未建構的槽4，立即破壞表示。

邏輯 k	舊槽 (2+k) mod 4	值	新槽
0	2	C	0
1	3	D	1
2	0	E	2
3	1	F	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

```
allocate new capacity C'
for k=0..size-1: construct new[k] from old[slot(k)]
commit new storage; head=0; C=C'; destroy/release old storage
```

提交後邏輯值仍 C,D,E,F；append G 寫新槽4。擴容單次 Θ(size)，倍增且僅在滿時成長，從空開始入列可按相同幾何級數得攤銷 Θ(1)。固定容量即時系統若要求最壞常數，不能把這項擴容悄悄包含在內。

出列搬移及空滿表示的獨立比較

同樣刪除 A，實體槽位的更新不同

線性出列：搬移 B、C



環狀出列：B、C 保留原槽



容量 4；— 表示無活資料。空與滿可同為 head=0，以 size=0 / 4 區分。

固定容量4時，空與滿可能有相同 head；必須用 size 或另一套明確約定區分。線性左移版出列要搬移所有剩餘值，環狀版只推進 head 並銷毀原首物件，兩者保持相同 FIFO 抽象值，但成本不同。

迷宮需要明訂狀態與合法移動

- 迷宮格子是頂點，相鄰可通行格之間有邊。此例允許上下左右四向移動，不允許斜走。
- 搜尋狀態至少包含所在格、已訪問資訊，以及重建路徑所需的 predecessor。
- DFS 可用堆疊，找到一條路徑但通常不保證步數最少。
- 若所有移動成本都是 1，BFS 使用佇列，首次到達出口的層數就是最短步數。

DFS 回溯與迷宮原創例子

以 # 表示牆，座標從左上角 (0,0) 起算。

```
S . # .
# . # .
. . . G
```

- 固定鄰居次序為右、下、左、上。從 S=(0,0) 可走 (0,1)→(1,1)→(2,1)→(2,2)→G。
- 堆疊框架可存 (cell,next_direction)。遇到死路便 pop，回到上一格繼續尚未嘗試的方向。
- 已訪問標記避免在圖中重複繞圈。若要列舉所有簡單路徑，回溯時的標記策略必須另外設計。

迷宮 DFS 的可恢復框架

若只把目前格放堆疊而不記下一方向，回溯後可能重做鄰居掃描。框架 (u,next_direction) 保存遞迴的程式位置，visited 在首次推入時設定。

```
visited[start]=true; parent[start]=none; push((start,0))
while stack 非空:
    (u,i)=top()
    if u==goal: return parent path
    if i==4: pop(); continue
    top().next_direction=i+1
    v=neighbor(u,i)                  // 右、下、左、上
    if v 在界內且非牆且 not visited[v]:
        visited[v]=true; parent[v]=u; push((v,0))
return unreachable
```

本章格圖在首次走到 (2,1) 時，右鄰 (2,2) 會先被推入，再往右找到 (2,3)=G；路徑共5邊。若不在首次抵達 G 時提早停止，完整 DFS 發現次序為 (0,0),(0,1),(1,1),(2,1),(2,2),(2,3),(1,3),(0,3),(2,0)。每一框架最多嘗試四方向；有限格數且只首次推入，保證終止。parent 邊沿發現時間嚴格回退，因此不成環並能還原合法路徑。

迷宮 BFS 的距離與前驅實例

仍用同一格圖，BFS 在入列時設定 dist 及 parent。所有邊成本1，距離如下；牆保留 #。

```
dist[start]=0; parent[start]=none; enqueue(start)
while queue 非空:
    u=dequeue()
    for v in 四向合法鄰居(u):
        if v 尚未發現:
            dist[v]=dist[u]+1; parent[v]=u; enqueue(v)
```

```
0  1  #  7
#  2  #  6
4  3  4  5
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

迷宮 BFS 的距離與前驅實例 （續）

出列格	新發現格（距離）	出列後佇列
(0,0)	(0,1):1	[(0,1)]
(0,1)	(1,1):2	[(1,1)]
(1,1)	(2,1):3	[(2,1)]
(2,1)	(2,2):4、(2,0):4	[(2,2),(2,0)]
(2,2)	(2,3):5	[(2,0),(2,3)]
(2,0)	無	[(2,3)]
(2,3)	(1,3):6	[(1,3)]
(1,3)	(0,3):7	[(0,3)]
(0,3)	無	[]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

迷宮 BFS 的距離與前驅實例 (續)

由 G 的 parent 逆追 `(2,2),(2,1),(1,1),(0,1),(0,0)`，反轉得到5邊最短路。首次發現時，所有更小層已先處理，這是最短性理由，不能只因用了 queue 就宣稱正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋正確性與成本

- 不變量為每個被探索的格都從起點可達，predecessor 邊連回較早發現的格。
- DFS 若探索所有合法鄰居且沒有未處理的堆疊框架，就已涵蓋所有可達格；出口未被發現則不可達。
- 有 `V` 個可通行格、`E` 條相鄰邊時，鄰接串列模型的 DFS/BFS 為 `Θ(V+E)`，標記與 predecessor 空間 `Θ(V)`。
- 四鄰格子中 `E=O(V)`，所以是線性成本。若每步複製整條路徑，實作可能超出此界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

運算式表示法與語法

- 中序 `a+b*c`、前序 `+ a * b c`、後序 `a b c * +` 代表相同的運算樹。
- 本章先限定二元運算子、整數 token、括號與明訂優先序。負數 token 與一元負號是不同的語法設計。
- 中序需要優先序與結合性。`a-b-c` 預設左結合，代表 `(a-b)-c`。
- 空白不能可靠地切分所有實際程式語言。完整編譯器需先詞法分析，再處理語法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後序求值的運算元次序

1. 從左到右掃描 token。數值直接 push。
2. 遇到二元運算子，先 pop 右運算元 `b`，再 pop 左運算元 `a`，計算 `a op b` 後 push。
3. 若不足兩個值即拒絕。掃描完成後堆疊必須恰好剩一個值。

原創例子 `8 3 2 * -` 的堆疊依序為 `[8]`、`[8,3]`、`[8,3,2]`、`[8,6]`、`[2]`。除法還需檢查零除與算術範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前序求值與線性時間論據

- 前序求值可由右到左掃描。遇到運算子時，第一次 pop 是左運算元，第二次是右運算元。
- 原例 `- 8 * 3 2` 由右向左算出 `3*2=6`，最後 `8-6=2`。
- 每個 token 至多 push、pop 固定次數，所以中序轉後序及前後序求值皆為 $O(t)$，t 為 token 數。
- 輔助堆疊最壞 $O(t)$。這裡把固定範圍算術視為常數時間，任意精度數值另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中序轉後序的完整程序

本節文法限定非負整數 token、二元 `+ - * / ^` 與括號。`^` 是本教學文法的右結合冪次，非 C++ 的 XOR；其餘運算子左結合。優先序依序為 `^`、`*/`、`+-`。一元負號不在此文法內。

```
output = []; ops = []; need_operand = true
for token in tokens:
    if token 是數值:
        require need_operand; output.append(token); need_operand=false
    else if token == '(':
        require need_operand; ops.push('(')
    else if token == ')':
        require not need_operand
        while ops 非空且 ops.top != '(': output.append(ops.pop())
        require ops 非空; ops.pop()      // 丟棄配對左括號
        need_operand=false
    else if token 是二元運算子 op:
        require not need_operand
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中序轉後序的完整程序 （續）

```
        while ops 非空且 ops.top != '(' 且
              (prec(ops.top)>prec(op) 或
               (prec(ops.top)==prec(op) 且 op 左結合)):
            output.append(ops.pop())
        ops.push(op); need_operand=true
    else: reject
require not need_operand                  // EOF，拒絕空式或缺最後運算元
while ops 非空:
    require ops.top != '('               // 未配對左括號，拒絕
    output.append(ops.pop())
return output
```

while 不能改成 if：新運算子可能同時使多個較高或同級左結合運算子出棧。EOF 不是一般數值 token，必須另作清空與括號驗證。所有 require 失敗皆拒絕，不交出不完整後序式。

EOF 清空不能省略

9-4

讀入 token	運算子堆疊（底→頂）	已輸出
9	∅	9
-	-	9
4	-	9 4
EOF	∅	9 4 -

EOF 前只有 9 4，不是完整後序式。EOF 彈出 – 得 9 4 –，值為5；前序為 – 9 4。

括號子式完整掃描

(9-4)*(2+3)

讀入 token	運算子堆疊（底→頂）	已輸出
(	(	∅
9	(	9
-	(-	9
4	(-	9 4
)	∅	9 4 -
*	*	9 4 -
(	* (	9 4 -
2	* (	9 4 - 2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

括號子式完整掃描 （續）

讀入 token	運算子堆疊（底→頂）	已輸出
+	* (+	9 4 - 2
3	* (+	9 4 - 2 3
)	*	9 4 - 2 3 +
EOF	∅	9 4 - 2 3 + *

後序求值堆疊為 [9]→[9,4]→[5]→[5,2]→[5,2,3]→[5,5]→[25]。前序 * - 9 4 + 2 3 也算得25。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連續運算子的反覆彈出

9-4*2/1+3

讀入 token	運算子堆疊（底→頂）	已輸出
9	∅	9
-	-	9
4	-	9 4
*	- *	9 4
2	- *	9 4 2
/	- /	9 4 2 *
1	- /	9 4 2 * 1
+	+	9 4 2 * 1 / -

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連續運算子的反覆彈出 (續)

讀入 token	運算子堆疊（底→頂）	已輸出
3	+	9 4 2 * 1 / - 3
EOF	∅	9 4 2 * 1 / - 3 +

讀 + 時先彈 /，再彈同級左結合 -；只彈一次會錯。正確值 9-(4×2)/1+3=4。右結合 2^3^2 則在第二個 ^ 不彈同級，後序為 2 3 2 ^ ^，值512，不是64。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不合法 token 序列的拒絕點

輸入	失敗時刻	拒絕理由
`(9-4`	EOF	彈 − 後仍有 (，左括號未配對
`9-4)`	讀)	彈 − 後無 (可配
`9-`	EOF	need_operand 仍為真
`9 4`	讀第二個數值	缺二元運算子
`()`	讀)	括號內缺運算元

不可用堆疊值數量猜一元或二元語法。求值另拒絕除零與固定整數溢位。

運算子堆疊的不變量與結束證明

把尚未配對的左括號視為堆疊分段邊界。每一分段內，已堆疊運算子由底到頂的優先序非遞減；相等優先序只有等待右結合運算時可同時保留。輸出是已確定子運算式的後序排列；堆疊運算子尚在等待右運算元或更高優先子式。

讀入新 op 時反覆 pop，恰好把語法上應先完成的運算子輸出。右括號把該括號內全部待辦運算輸出，保留外層的運算子；EOF 則不再有未讀運算元，必須將全部待辦輸出。若還有左括號，文法就沒有合法配對，不能當運算子輸出。

每個運算子至多推入一次、彈出一次，括號亦如此。雖然某一 token 觸發多次 pop，整段 t 個 token 仍 O(t) 時間、O(t) 空間。這是跨 token 的總量論證，不是宣稱每次迴圈皆最壞常數。剖析負整數 token、一元運算、函式呼叫與浮點特殊值，需擴充文法及對應正確性，不在此二元版默認支援。

來源與授課邊界

練習題與推導答案分別見本章獨立檔案。

標準容器介面核對 C++ stack 與 queue。搜尋理論對照 MIT 6.006 2020 L9–L10。迷宮與運算式例子為原創。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study